Key Classes and Responsibilities

1. MainActivity

- **Inheritance:** AppCompatActivity

- **Responsibilities:**
    - Launches the app and determines the start destination (SETUP or HOME).
    - Hosts fragments such as SetUpFragment and HomePageFragment.

- **Key Methods:**
    - onCreate(Bundle)
    - showSetUpFragment()
    - showHomePageFragment()

2. Fragments

All fragments inherit from Fragment or DialogFragment.

- **SetUpFragment**
    - Handles account creation and user role selection.
    - Uses Firebase Authentication and Firestore.
    - Key Methods: attemptCreateAccount(), navigateToHome()

- **HomePageFragment**
    - Displays the home page content.

- **CreateEventFragment**
    - Handles creating or editing events.
    - Uses EventInput and ValidationContext to validate inputs.

- **WaitlistManagementFragment**
    - Manages the waiting list of an event.
    - Uses EntrantAdapter to display users.
    - Integrates with LotterySystem for drawing entrants.

- **EventOverviewFragment**
    - Implements WaitingListDialogListener to handle waiting list actions.

- **NotificationsFragment**
    - Displays user notifications using NotificationsAdapter.

- **UserProfileFragment**

    - Displays and updates user profile information.

3. Models

- **User / Admin / Organizer / Entrant:** Represents the different types of users.
- **Event:** Contains event details such as title, description, registration dates, and waiting list capacity.
- **Notification:** Represents notifications with type, status, and timestamps.

4. Repositories

- **NotificationRepository:** Handles create, read, update and delete operations on notifications.
- **WaitingListRepository:** Handles joining, leaving, and retrieving waiting lists.

5. Adapters

- **EventAdapter**
- **NotificationsAdapter**
- **WaitlistManagementFragment & EntrantAdapter:** Follows the **Adapter Pattern** to convert data models into UI components.

6. System / Utilities

- **LotterySystem**
    - Manages lottery draws and invitation handling.
    - Uses Firebase Firestore and NotificationRepository.
    - Acts as a singleton like utility.

- **ValidationResult / ValidationContext**
    - Encapsulates validation logic for event creation and editing.

- **MockEventData**
    - Provides sample events for testing.

Design Patterns Observed

1. **Adapter Pattern:** Used by all adapter classes (EventAdapter, NotificationsAdapter, EntrantAdapter) to map data models to UI views.

2. **Observer Pattern:** Implemented via WaitingListDialogListener interface for communication between DialogFragments and hosting fragments.

3. **Singleton / Utility:** LotterySystem behaves as a singleton for managing event lotteries and notifications.

4. **Factory Methods:** Fragments like CreateEventFragment and WaitingListDialogFragment provide static newInstance( ) methods for object creation with parameters.

Relationship Notes

- **Inheritance**
    - Activities inherit from AppCompatActivity.
    - Fragments inherit from Fragment or DialogFragment.
    - Adapters inherit from Adapter or ArrayAdapter.

- **Composition / Aggregation**
    - Fragments compose adapters and repositories.
    - Adapters aggregate lists of User or Notification.

- **Dependencies**
    - Fragments depend on repositories (NotificationRepository, WaitingListRepository) and systems (LotterySystem) via dotted arrows in UML diagrams.

- **Interfaces**
    - WaitingListDialogListener allows decoupled communication between DialogFragments and fragments.